Building a Professional Browser-Based Word Processor: A Solo Founder's R&D and Implementation Plan

Executive Summary

Build on **Tiptap** (the MIT-licensed React framework wrapping ProseMirror), not from scratch and not on Lexical/Slate/Quill. For your hard pagination requirement, adopt Tiptap's official **Pages** Pro extension as the primary path, with a **Paged.js**-based server pipeline for print/PDF as the fidelity backstop. ProseMirror's schema-based, decoration-capable, battle-tested model is the only open-source foundation that simultaneously satisfies: (a) Word-like structured formatting, (b) real pagination, (c) Yjs collaboration, (d) tracked changes, and (e) DOCX round-trip — all with first-class React/Next.js integration.

The single most important strategic truth: **true page-based WYSIWYG in the browser is the hardest part of this project, has no perfect open-source solution, and every approach has sharp tradeoffs.** Google Docs solved it only by abandoning the DOM entirely for a custom canvas layout engine. You should not attempt that as a solo builder. Instead, accept a pragmatic two-tier strategy: ProseMirror-based visual pagination for editing, and a CSS Paged Media (Paged.js + headless Chromium) pipeline for pixel-accurate print/PDF output.

This report covers all 12 feature areas, recommends a concrete stack, gives a phased solo roadmap, and is explicit about where fidelity ceilings exist (DOCX round-trip especially).

Feature-by-Feature Breakdown

1. Clipboard & Basic Editing (Cut/Copy/Paste/Format Painter)

Data model: Cut/copy/paste operate on ProseMirror `Slice` objects — a fragment plus "open depth" on each side describing how nodes are cut through. ProseMirror already wires `DOMParser.fromSchema` as the default clipboard parser, so pasted HTML is mapped into your schema via each node/mark's `parseDOM` rules. Anything not described by a parse rule is dropped — this is your first, automatic layer of sanitization.

Browser API reality: Use the **Async Clipboard API** (`navigator.clipboard.read()/write()`), the modern, non-blocking, permissions-aware replacement for the deprecated `document.execCommand()`. Hard constraints to design around:

- **Secure context only:** `navigator.clipboard` is undefined on HTTP pages (works on HTTPS and localhost). On Vercel you're HTTPS by default.
- **User-gesture requirement:** Writes must occur inside a user gesture (e.g., click handler). Programmatic writes outside a gesture fail.

- **Read permission:** `read()/readText()` prompts the user and is often gated on document focus / proximity to a user interaction. Provide a fallback "Paste" button. Amitse
- **Browser sanitization inconsistency:** Chromium strips `<script>` and dangerous content, and strips meta tags from `text/html` on the async API (but keeps them via legacy `setData/getData`) — a documented interoperability gap (W3C clipboard-apis issue #150) that breaks "infer source from meta tag" logic. Don't rely on clipboard meta tags.

Rich text / image / HTML paste pipeline:

1. Intercept `paste` via ProseMirror's `transformPastedHTML` / `handlePaste`.
2. Run pasted HTML through **DOMPurify** before it reaches `parseDOM` (defense-in-depth; ProseMirror's schema parse is not a security boundary by itself).
3. Offer "Paste" vs "Paste without formatting" (the latter takes `text/plain`).
4. For pasted images (`image/*` blobs from `clipboard.read()`), upload to Supabase Storage and replace with a stored URL rather than embedding huge base64 in the doc.

Format Painter: Implement as a transient "stored marks + paragraph attrs" clipboard. On "copy format," capture the set of marks (and block attributes like alignment/style) at the cursor; on the next selection apply `addMark`/`setBlockType` with those captured values. Double-click = sticky mode (stays active until Esc), mirroring Word.

Save/export preservation: Because formatting lives as marks/attrs in the ProseMirror JSON, it round-trips losslessly to your own storage; fidelity loss only happens at DOCX/PDF boundaries (see Export strategy).

2. Font Tools

All character-level formatting is modeled as **ProseMirror marks** (inline annotations on text ranges), applied with `toggleMark`/`addMark` over the current selection:

- **bold, italic, underline, strikethrough** → standard marks (`strong`, `em`, `u`, `s`), each with `parseDOM`/`toDOM`.
- **subscript/superscript** → mutually exclusive marks (toggling one removes the other).
- **font family / font size / font color / highlight color** → marks carrying an attribute (`textStyle` mark with `fontFamily`, `fontSize`, `color` attrs; `highlight` mark with `color`). Tiptap ships `@tiptap/extension-text-style`, `color`, `highlight`, `font-family` as MIT extensions.
- **increase/decrease font size** → command reads current `fontSize` attr, steps through a ramp (8,9,10,10.5,11,12,14…), re-applies the mark.
- **change case** (UPPER/lower/Title/Sentence/tOGGLE) → NOT a mark; it transforms the actual text content. Walk the selection's text nodes and `replaceWith` transformed strings in a single transaction. (Word also does this as a text transform, not formatting, which is why it doesn't "remember" the original case.)

- **clear all formatting** → `removeMark` for all marks across selection + reset block attrs to the default ("Normal") style.
- **advanced effects (shadow, outline, glow, small caps, letter-spacing, decorative):** model as a `textStyle` mark with extra attributes serialized to inline CSS (`text-shadow`, `-webkit-text-stroke`, `font-variant: small-caps`, `letter-spacing`). **Fidelity caveat:** these are CSS-only; they will NOT survive DOCX export cleanly (Word's text-effects model differs), and outline/glow have no DOCX equivalent. Flag in UI as "screen/PDF only."

3. Paragraph Tools

Block/paragraph formatting is modeled as **node attributes** (not marks) and **node wrapping** (for lists):

- **alignment, line spacing, paragraph spacing (before/after), shading, borders, indents** → attributes on the `paragraph`/`heading` node (`textAlign`, `lineHeight`, `spacingBefore`, `spacingAfter`, `background`, `borders`, `indent`). Applied via `setBlockType`/`updateAttributes` over the selected block range.
- **bulleted / numbered / multilevel lists** → wrapping nodes: `bulletList`/`orderedList` containing `listItem` nodes. Multilevel = nesting list nodes; indent/outdent = `sinkListItem`/`liftListItem` (ProseMirror's list commands enforce valid nesting via schema content expressions, the key defense against "broken outline nesting").
- **decrease/increase indent** → on list items use sink/lift; on plain paragraphs adjust the `indent` attr (clamped 0–N).
- **sort alphabetically/numerically** → operate on sibling block nodes or list items: extract text keys, sort, rebuild the fragment in one transaction.
- **show/hide formatting marks** (pilcrows, spaces, tabs, breaks) → a **decoration**-based overlay (widget/inline decorations) toggled by a plugin; never mutate the document — decorations "influence the way the document is drawn, without actually changing the document." Prosemirror

Preventing broken lists: Rely on schema **content expressions** (e.g., `bulletList: { content: "listItem+" }`) compiled into finite automata that validate structure on every transaction, plus ProseMirror's list commands. This is exactly why a schema-based engine (ProseMirror) beats a freeform one (Slate) for Word-like structure.

4. Styles System (Style Gallery)

Model styles as **named presets** that resolve to node attrs + marks, stored in a `styles` map in document metadata (and optionally per-user/global in Supabase). Approach:

- Each block node carries a `styleId` attr (`"Normal"`, `"Heading1"`, `"Quote"`, `"Title"`, `"Code"`, or a custom UUID).

- A style definition = `{ basedOn, blockAttrs, marks, nextStyle }`. **Inheritance** is resolved by walking the `basedOn` chain (Word's "based on" model). "Normal" is the root.
- **Rendering:** A ProseMirror plugin maps `styleId` → CSS class; the resolved CSS lives in a `<style>` block generated from the style table (so "update style" = regenerate CSS + optionally rewrite affected nodes).
- **Update style across document:** changing a style definition instantly restyles every node with that `styleId` (CSS-driven) — no document mutation needed for pure formatting. For attributes that must live in the model (e.g., numbering), run a sweep transaction.
- **Theme support:** a theme = a palette + font scheme referenced by styles (CSS custom properties: `--theme-heading-font`, `--theme-accent`). Switching themes re-resolves variables.

This mirrors Word/Docs: heading styles are *semantic* (`Heading 1` → `h1`), which also makes DOCX export via style-mapping clean (mammoth's recommended pattern is `p[style-name='Heading 1'] => h1`). GitHub

5. Editing Tools (Find/Replace/Select)

- **Find/Replace** → a plugin that scans document text, builds **decorations** to highlight matches (no mutation), and runs `replace` transactions on confirm. Support case-sensitive, whole-word, and **regex** (compile user pattern, guard against catastrophic backtracking with a timeout).
- **Search inside comments/tables** → because everything is nodes in one tree, recursive `descendants()` traversal naturally reaches table cells; comments stored as marks/threads are scanned separately.
- **Select All / Select Text** → ProseMirror selections (`TextSelection`, `AllSelection`).
- **Select Objects / Select Similar Formatting** → custom selection logic: "select objects" filters to floating/leaf nodes (images, shapes) and drives the Selection Pane; "select similar formatting" reads marks/attrs at cursor and multi-selects matching ranges (decoration-highlighted, since ProseMirror has one selection — emulate multi-select via decorations + a command target list).
- **Fast search in large docs:** maintain an incrementally-updated plain-text index (rebuild only changed ranges from transaction steps); for 100k+ word docs, debounce and chunk scanning in `requestIdleCallback`. ProseMirror fixed quadratic display-update complexity for documents with thousands of child nodes in prosemirror-view 1.8.9 (April 2019), so the editor core scales; your search layer must too. Grokipedia

6. Page Setup & Pagination (PRIORITY)

This is the make-or-break area. The four real-world approaches, compared honestly:

Approach A — Google Docs' custom canvas layout engine. Per Google's Workspace Updates blog (May 11, 2021): "Over the course of the next several months, we'll be migrating the underlying technical implementation of Docs from the current HTML-based rendering approach to a canvas-based approach to improve performance and improve consistency in how content appears across different platforms." Google maintains a "side DOM" + ARIA live regions purely for accessibility. Steve Newman, co-founder of Upstartle/Writely (acquired by Google in 2006), wrote on Hacker News that online word processors "have extremely specific requirements for layout, rendering, and incremental updates" the DOM isn't always suited to meet. **Verdict for you: do not attempt.** This is multi-team, multi-year engineering and sacrifices accessibility and ecosystem. The New Stack

Approach B — ProseMirror/Tiptap pagination plugins (visual pages while editing).

- **Tiptap Pages (official Pro extension, alpha):** Provides A4/Letter/Legal/Tabloid/A3/A5/custom formats, visual page breaks, dynamic headers/footers with `{page}`/`{total}` placeholders and first-page/odd-even slots, edit-in-place headers, zoom in [0.25, 4], and content that flows across pages. Requires companion packages: `@tiptap-pro/extension-convert-kit` (schema), `@tiptap-pro/extension-pages-tablekit` (pagination-safe tables), `@tiptap-pro/extension-pages`. Tiptap + 3
  - **Critical documented limitation:** "Non-splittable blocks larger than a page cause an infinite layout loop. This is the single most important limitation to know about before adopting Pages." A table row, figure, or positioned block taller than the page content area "keeps trying to move it forward and never stabilises… The result is an unresponsive editor." Mid-row table splitting is on the roadmap, ETA TBD. You must cap block/row heights (`max-height`) and validate imported DOCX. Tiptap + 2
  - **License/cost:** Pro extension — requires a Tiptap subscription. Editor core is MIT/free; per tiptap.dev/pricing and Tiptap's "new pricing model is live" release notes, Cloud Platform tiers are **Start $49/mo (500 cloud docs), Team $149/mo (5,000 docs), Business $999/mo (50,000 docs)**; the free plan was removed in June 2025, replaced by a 30-day trial.
- **Community/OSS options:** `tiptap-extension-pagination` (hugs7, MIT), `tiptap-pagination-plus`, and `@adalat-ai/page-extension` (MIT) implement pagination by measuring node heights (`getBoundingClientRect`) and inserting spacer/decoration page breaks. These are free but rougher: developers report paragraphs not splitting mid-block at exact Google-Docs-like positions, padding/placement bugs, and CPU cost recomputing on every keystroke (each char can have different width; font-size varies). The core hard problem is splitting a paragraph at the exact character where it overflows — browsers give per-character width variance. Medium

Approach C — Paged.js / CSS Paged Media (print & PDF fidelity). Paged.js (MIT, Coko Foundation) is a polyfill for the W3C CSS Paged Media + Generated Content specs. It fragments HTML into pages using CSS columns, honors `@page { size; margin; bleed; marks }`, generates

running headers/footers and `counter(page)`, and shows a paginated preview in-browser. It is "still experimental," works best in Chromium, and **cannot use CSS variables for page size**; Firefox needs manual PDF sizing because `@page { size }` support varies. This is the right tool for **export/print**, not live editing (re-fragmenting on every keystroke is too slow). Pagedmedia + 2

Approach D — Pure print CSS + browser print. `@media print` + `break-before/inside` + `@page`. Cheapest, but no in-editor page view and inconsistent cross-browser pagination. Acceptable only for an MVP "print" button.

RECOMMENDED PAGINATION ARCHITECTURE (concrete):

1. **Editing view:** Tiptap Pages (if budget allows the subscription) for true page-aware editing; otherwise start with the MIT `tiptap-extension-pagination` and accept rougher break behavior. Render visible page boundaries, margins, headers/footers, zoom.
2. **Print/PDF view:** Serialize the ProseMirror doc → clean semantic HTML + a generated print stylesheet → run **Paged.js in headless Chromium (Puppeteer/Playwright) server-side on a Node function** → `page.pdf()`. This gives accurate page boxes, margins, running heads, page numbers, and selectable text. Server-side guarantees output independent of the user's screen/browser.
3. **Section/column/line-number/hyphenation settings** live as attributes on `section` nodes (a top-level block grouping). Each section maps to `@page` named rules in the generated stylesheet (different margins/orientation/columns per section). CSS `columns`, `column-break`, and `hyphens: auto` cover columns/hyphenation; line numbers via CSS counters in the print pipeline.
4. **Responsive zoom:** CSS `transform: scale()` on the page container (cheap, doesn't reflow), with the editor measuring at scale 1.

Known limitations to disclose: in-editor pagination will never be 100% identical to the PDF output (two different layout engines); you must reconcile by using the same page metrics (96 DPI: Letter = 816×1056px, A4 = 794×1123px) and the same margins in both. Headers/footers, exact widow/orphan control, and mid-row table breaks are the usual divergence points.

7. Paragraph Layout Tools

Implement as paragraph node attributes: `indentLeft`, `indentRight`, `indentFirstLine`, `indentHanging`, `spacingBefore`, `spacingAfter`. Hanging indent = positive left indent + negative first-line indent (model both; render via `padding-left` + `text-indent`).

The three formatting levels (explain in UI and docs):

- **Text/character formatting** = marks on inline ranges (bold, font, color). Smallest scope.
- **Paragraph formatting** = attributes on block nodes (alignment, spacing, indents, list membership). Scope = whole paragraph(s).

- **Page/layout formatting** = section/page attributes (margins, orientation, columns, paper size). Scope = section/document.

Keeping these in distinct layers (marks vs block attrs vs section attrs) is what prevents the "everything is inline CSS soup" failure mode and makes styles, DOCX mapping, and pagination tractable.

8. Object Arrangement (Floating Objects Layer)

This is the second-hardest area after pagination, because **floating, text-wrapped, z-ordered objects fundamentally conflict with a linear document tree.**

Recommended model — a hybrid two-layer system:

- **Inline/anchored objects** (default images): regular leaf nodes in the flow (`image` node), rendered via a **NodeView** that adds resize handles. Simple, paginates naturally.
- **Floating objects** (text-wrapped images, shapes, text boxes, charts): store as a node **anchored to a paragraph** but carrying `floating` attrs: `{ x, y, anchorMode: 'page'|'paragraph', wrap: 'square'|'tight'|'topBottom'|'behind'|'front', z, rotation, flipH, flipV }`. Render via a NodeView that is absolutely positioned (`position:absolute`) relative to the page container. Text wrap is approximated with CSS `shape-outside` + float for square/tight wraps; "behind/in front of text" uses z-index layers.
- **Object layer mechanics:** bring forward/backward/to-front/to-back = mutate `z`; align/distribute = compute bounding boxes and update `x/y` for the selected set; group/ungroup = a `group` wrapper node holding child object refs; rotate/flip = CSS `transform`. The **Selection Pane** is a React panel listing all object nodes (from a `descendants()` scan) with visibility toggles and z-order drag.

Coexistence with paginated flow (the hard part): Floating objects anchored to `page` coordinates must be re-anchored when content reflows across pages. Practical compromise (what most web editors accept): anchor floats to a paragraph and let them move with it; offer "fix on page" only in the export/print pipeline (Paged.js page floats / CSS `float` in `@page`). Be explicit that **arbitrary Word-style tight text-wrap around irregular shapes is not fully achievable in HTML** — `shape-outside` handles convex shapes only. ProseMirror NodeViews can render absolutely-positioned, draggable overlays (community examples exist for post-it-style absolutely-positioned nodes), but you own the wrap/reflow logic.

9. UI / UX Research

Comparison of paradigms:

- **Ribbon UI** (Word): best for *feature-dense, discoverability-first* apps. Microsoft's own guidelines say the ribbon "MUST NOT coexist with top-level menus and toolbars," must sit at the top spanning full width, and should completely disappear when the window is under

~300×250px. Tabs should NOT auto-switch on document selection (except contextual tabs). Pros: scales to hundreds of commands, familiar to Word users. Cons: heavy, screen-hungry, overkill for casual use. Actipro Software

- **Compact toolbar** (Google Docs): single row + overflow; lighter, faster, less discoverable for advanced features.
- **Floating/bubble toolbar** (Medium/Notion): appears on selection; great for focus/mobile, hides advanced tools.
- **Slash commands** (Notion): fast for power users inserting blocks; poor discoverability for formatting.
- **Contextual menus / mini-toolbar** (Word's on-selection mini bar): complements any primary UI.

RECOMMENDATION: A **ribbon as the primary desktop UI** (you're explicitly cloning Word), with **contextual mini-toolbar on selection** and **slash commands** as accelerators. This matches user expectations for a "Word-like" product and is the best fit for the dense feature set in this spec.

Layout recommendation (decision-ready):

- **Desktop:** Top = ribbon (collapsible tab groups: Home, Insert, Layout, References, Review, View) + Quick Access Toolbar. Left = collapsible navigation/outline + page thumbnails. Center = document canvas with ruler + zoom. Right = contextual inspector (formatting / object / comments / version history as switchable panels). Bottom = status bar (page X of Y, word count, zoom slider, language).
- **Tablet:** Collapse ribbon to a single contextual bar that can expand to "ribbon world" on tap; right inspector becomes a slide-over. (This convergent toolbar↔ribbon pattern keeps one UI across desktop/tablet.)
- **Mobile:** Abandon the ribbon. Use a floating selection toolbar + a bottom sheet for formatting + slash/plus insert menu. Read-optimized; editing is secondary.

10. Technical Architecture (Editor Engine Comparison)

Engine	License	Model	Pagination	Collab	Decorations	Verdict for this project
ProseMirror	MIT	Schema'd node/mark tree	Plugins exist (hard)	y-prosemirror (Yjs)	**Yes (pure)**	Best foundation; low-level
Tiptap	MIT (core)	ProseMirror	**Official Pages (Pro)**	Yjs + Pro	Yes	**RECOMMENDE** — PM power + Re DX

Engine	License	Model	Pagination	Collab	Decorations	Verdict for this project
Lexical	MIT	Node tree, batched	None native; lacks pure decorations	Yjs	**No pure decorations** (decorator nodes mutate doc)	Fast, but decorati gap hurts collab cursors/search/tr changes
Slate	MIT	Custom JSON (you define)	None	Yjs	Limited	Max flexibility, slower, fewer plugins, you build everything
Quill	BSD/MIT	Delta (OT)	None	y-quill	No pure decorations	Legacy feel; Delta model dated for Word-like structu
Editor.js	Apache-2.0	Block JSON	None	Weak	No	Block CMS, not a word processor
TinyMCE	**GPL or commercial**	DOM/HTML	Limited	Paid	—	Licensing cost; HTML-soup mod
CKEditor 5	**GPL or commercial**	Custom model	Pagination = paid feature	Paid (OT)	Yes	Polished but commercial; body only DOCX round trip
Custom contenteditable	—	Yours	Yours	Yours	Yours	Only Google-scal teams should

Why Tiptap/ProseMirror wins for you:

- **Open-source requirement:** MIT core, self-hostable, no per-seat lock-in. (Lexical, Slate, Quill also MIT/BSD, but lose on the criteria below.)
- **Pagination requirement:** Tiptap is the *only* open-source-rooted option with an officially maintained pagination extension, plus a rich community of pagination plugins.
- **Pure decorations:** ProseMirror supports decorations that style/annotate without mutating the document — essential for collaborative cursors, search highlights, spell-check squiggles, and track-changes display. Lexical and Quill lack this; per Liveblocks, Lexical's "decorator nodes… mutate the content of the document," and an entire `quill-cursors` library exists just to work around Quill's gap.

- **Battle-tested + schema:** used by The New York Times, Asana; schema content-expressions prevent invalid structure (critical for lists/tables).
- **React/Next.js fit:** first-party `@tiptap/react`; integrates with your App Router stack. (Note the React perf caveat below.)

Liveblocks' 2025 editor survey ("Which rich text editor framework should you choose in 2025?") aligns: "the most well-rounded choice is Tiptap because it strikes a balance between being a feature-rich editor without being overly opinionated." On Lexical, after months building collab/comments/version-history: "although Lexical has a large community and commercial backing, it needs more time to mature before we can recommend it over Tiptap."

Sub-components:

- **Frontend framework:** Next.js 14+ App Router + TypeScript + Tailwind (your stack — good fit). The editor must be a **client component** (`"use client"`), dynamically imported with SSR disabled (contenteditable needs the DOM).
- **Document data model:** ProseMirror JSON (`{type, attrs, content, marks}` tree) as the canonical format. Store in Supabase Postgres as `jsonb`; store the Yjs binary update log separately for collab.
- **State management:** ProseMirror `EditorState` is the source of truth; mirror minimal UI state (active marks, page count) into React. Use `@handlewithcare/react-prosemirror` or Tiptap's React bindings; **memoize NodeViews** — naive React node views re-render every node after the cursor on each keystroke (position prop changes), so pass `getPos` not `pos` and `React.memo` aggressively.
- **Undo/redo:** ProseMirror `history` plugin (collab-aware; only undoes local changes).
- **Autosave:** debounced (500ms–2s) diff persistence of the JSON to Supabase + immediate Yjs update broadcast; Upstash Redis as the awareness/presence pub-sub and write buffer.
- **Offline:** Yjs IndexedDB provider (`y-indexeddb`) for local-first; syncs on reconnect via state-vector diffing.
- **Export/import & print/cloud sync/permissions:** see dedicated sections below.

11. Advanced Features

- **Real-time collaboration — CRDT (Yjs) over OT.** Use **Yjs + y-prosemirror**, server as relay/persistence (not ordering authority). Rationale: Yjs eliminates the entire class of OT transform bugs, gives instant local edits, linear (not quadratic) merge cost with offline duration, and has first-class ProseMirror bindings. **Honest caveat:** Yjs+ProseMirror has a known architectural wart — y-prosemirror can re-derive/replace the document representation on edits (issue #113 debate), and CRDT rich-text interleaving is an open research problem; some teams (e.g., Moment) rejected Yjs for these reasons. For a solo builder, Yjs is still the pragmatic best choice (vs. building OT). Transport: WebSocket

(Hocuspocus server, or Liveblocks/PartyKit managed). Presence/awareness via Redis pub-sub (Upstash). Beefed Moment

- **Comments:** anchored as marks (range) + thread store in Postgres; render in right panel.
- **Track changes / suggestions mode:** use `@handlewithcare/prosemirror-suggest-changes` (MIT) or `prosemirror-suggestion-mode` (MIT). These wrap `dispatchTransaction` so edits become insertion/deletion/modification **marks** instead of real mutations, with accept/reject commands and pilcrow decorations for paragraph-break changes. (Note ongoing debate about marks-vs-decorations for suggestions; marks are the shipped approach today.) npm GitHub
- **Version history:** snapshot JSON + Yjs update log; diff via `prosemirror-changeset`.
- **Templates:** stored documents cloned on creation.
- **Tables:** `prosemirror-tables` / Tiptap table extensions (use the **pagination-safe** TableKit if on Pages).
- **Images/shapes/charts:** object layer (§8). **Headers/footers:** page attrs + Pages extension or print pipeline. **Footnotes/endnotes:** footnote node with a sub-editor NodeView (classic ProseMirror footnote example) + CSS Paged Media footnote support in export. **TOC:** generated from heading nodes; Paged.js can build a cross-referenced TOC with page numbers. **Page numbers / bookmarks / hyperlinks:** counters in print pipeline; `link` mark (non-inclusive) for hyperlinks; bookmarks = anchored marks/IDs.
- **Spell check:** browser-native `spellcheck` attr is the cheapest. For custom UI/squiggles, use **nspell** (MIT, Hunspell-compatible, pure JS) or **nodehun** (MIT, Hunspell binding) with Hunspell dictionaries, rendered as decorations. GitHub
- **Grammar / AI writing / voice typing: LanguageTool** (self-host the LGPL Java server, or hosted API) for grammar; an LLM of your choice for AI assist via streaming; Web Speech API for voice typing.
- **Accessibility:** keep a real DOM (don't go canvas) so screen readers work; ARIA roles, full keyboard operability, focus management in the ribbon.

12. Security & Reliability

- **XSS from pasted/imported HTML:** sanitize with **DOMPurify** before parsing; never `dangerouslySetInnerHTML` raw mammoth/import output. **mammoth.js performs NO sanitization** — its docs warn it "should therefore be used extremely carefully with untrusted user input" (it can emit `javascript:` links and, with external file access on, exfiltrate server files). Always pipe mammoth output through DOMPurify and keep `externalFileAccess` off.
- **Clipboard security:** treat clipboard as untrusted ("toxic until the user explicitly asks to handle it"); explicit Paste button for reads; sanitize on paste; beware clipboard-hijacking (e.g., swapped wallet addresses) for sensitive fields. Fsjs Fsjs
- **Malicious document imports / file upload validation:** validate MIME + magic bytes, cap size, scan, convert in an isolated serverless function, reject DOCX with oversized/tall table

rows (would trigger the Pages infinite-loop). DOCX is a ZIP — guard against zip bombs.

- **Permissions/sharing:** enforce in Supabase **Row Level Security** (per-document ACL: owner/editor/commenter/viewer); never trust the client.
- **Encryption:** TLS in transit (Vercel), Postgres at-rest encryption (Supabase); consider per-doc encryption for sensitive tiers.
- **Autosave conflict handling:** Yjs merges concurrent edits; for non-collab saves use optimistic concurrency (version token) and last-writer-wins with a recovery snapshot.
- **Large-document performance:** see Performance strategy.
- **Browser compatibility:** Chromium-first (Paged.js + clipboard features are best there); test Firefox/Safari; provide graceful fallbacks (manual paste, server PDF instead of client print).

Recommended Editor Framework

Tiptap (MIT core) on ProseMirror (MIT), with Yjs for collaboration. Justification recap tied to your two hard constraints:

1. **Build on open source:** Tiptap's editor and all the primitives you need (schema, marks, decorations, tables, history, lists, suggestions) are MIT and self-hostable. You only pay Tiptap if you opt into the **Pages** Pro extension and/or their Cloud Conversion/Collaboration services — and even those have OSS or self-host alternatives.
2. **Pagination:** Tiptap is the only open-source-rooted editor with an officially maintained pagination extension (Pages), plus the richest community pagination ecosystem and pure-decoration support needed to render page guides without corrupting the model.

Cost-aware path: Start with a **100% MIT stack** (Tiptap core + community pagination + your own Paged.js PDF pipeline + self-hosted Yjs/Hocuspocus). Upgrade to Tiptap **Pages/Conversion** ($49 Start → $149 Team) only if/when the polish is worth it. Avoid TinyMCE/CKEditor: both are GPL-or-commercial, carry licensing cost at scale, and their HTML/custom-model round-trip is body-focused.

Best UI Layout

Ribbon-primary desktop (Home/Insert/Layout/References/Review/View tabs + Quick Access Toolbar), left outline+thumbnail panel, center canvas with ruler+zoom, right switchable inspector (Format/Object/Comments/History), bottom status bar. Contextual mini-toolbar on selection; slash commands as accelerators. Tablet collapses to a convergent expandable bar; mobile uses floating toolbar + bottom sheet (no ribbon). Follow Microsoft's ribbon rules (full-width top, no coexisting menus, collapse when tiny, no auto tab-switching except contextual tabs).

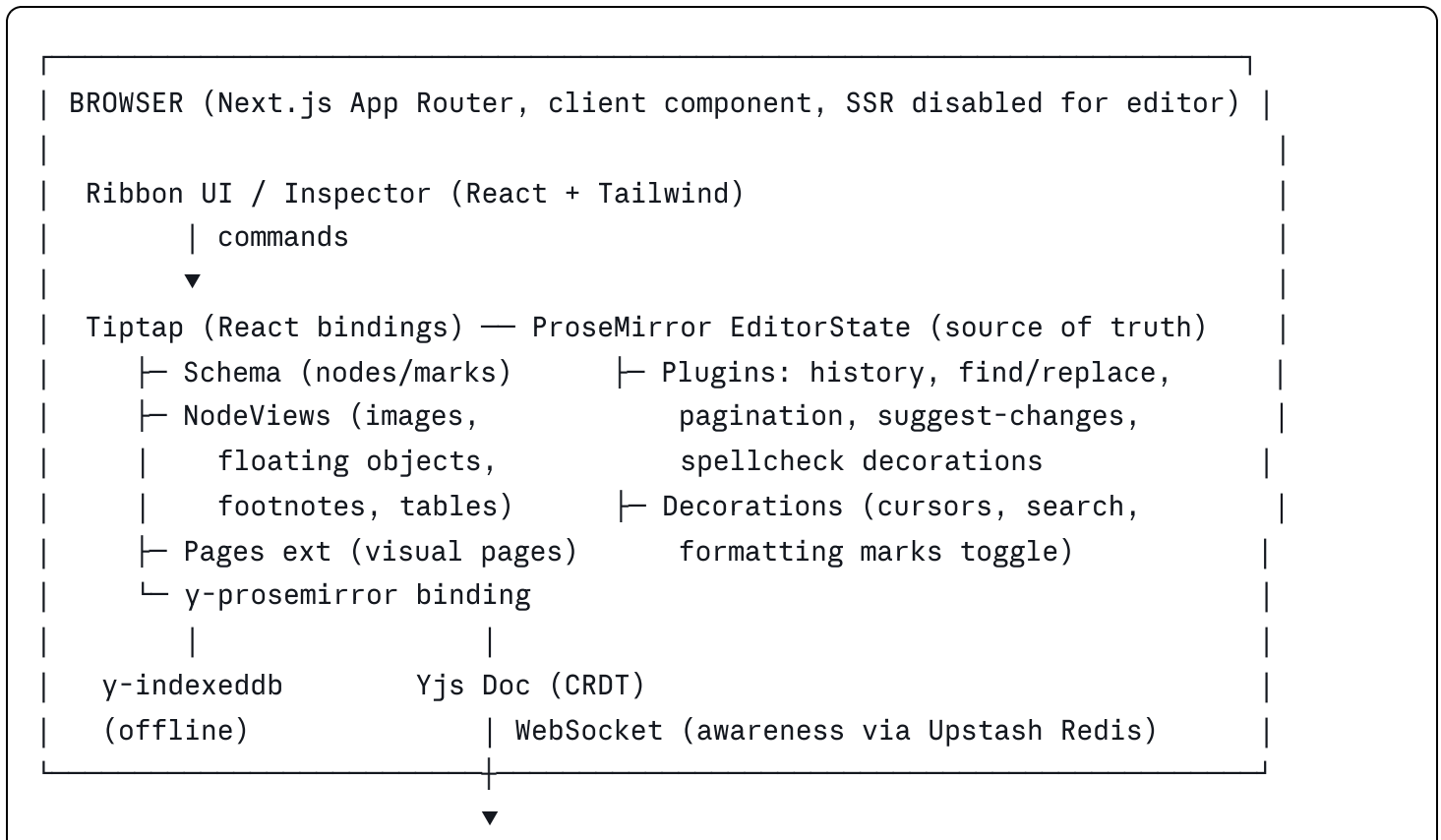
Data Model Recommendation

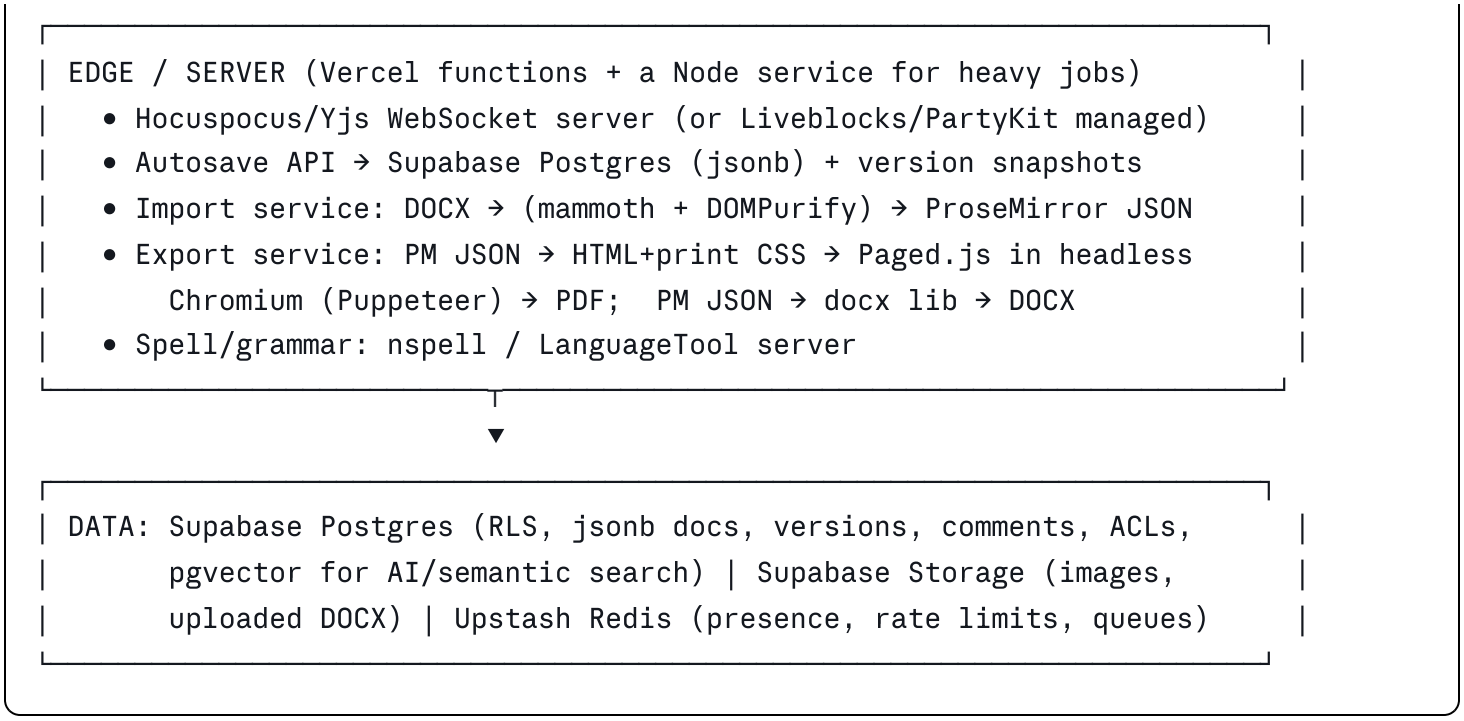
Canonical: ProseMirror JSON tree (`doc → section* → block* → inline*`, marks on inline).

- **Marks:** bold/italic/underline/strike/sub/sup/textStyle(font,size,color,effects)/highlight/link/comment/insertion/deletion/modification.
- **Block nodes:** paragraph, heading(level), bulletList/orderedList/listItem, blockquote, codeBlock, table/tableRow/tableCell, image(inline), floatingObject, footnote, pageBreak, sectionBreak.
- **Block attrs:** styleId, textAlign, lineHeight, spacingBefore/After, indentLeft/Right/FirstLine/Hanging, background, borders.
- **Section node attrs:** paperSize, orientation, margins{t,r,b,l}, columns, lineNumbers, hyphenation, headers/footers refs.
- **Document metadata:** styles table, theme, comment threads, properties.

Storage (Supabase): `documents(id, owner, jsonb content, ...)`, `doc_versions(doc_id, snapshot jsonb, created_at)`, `yjs_updates(doc_id, update bytea)`, `comments(...)`, `acls(...)` with RLS. Store images in Supabase Storage; keep base64 out of the JSON.

Technical Architecture Diagram (text form)





Implementation Roadmap (Solo Builder, Phased)

Phase 0 — Spike (1–2 weeks). Tiptap + Next.js client component working; basic marks; persist JSON to Supabase. Prototype pagination with the MIT `tiptap-extension-pagination` to feel the pain early. Decision gate: community pagination good enough, or budget for Tiptap Pages?

Phase 1 — Core editor (4–6 weeks). Full font/paragraph tools, lists (with schema-enforced nesting), styles system (Normal/Headings/Quote/Title/Code), find/replace with decorations, undo/redo, autosave, images (inline) via Supabase Storage, ribbon UI shell. Ship a usable single-user editor.

Phase 2 — Pagination & print/PDF (4–6 weeks). Lock in Pages (or community plugin); page setup (size/orientation/margins), section nodes, headers/footers, page numbers. Build the **server Paged.js → Puppeteer → PDF** pipeline. Reconcile editor metrics with print metrics. This is the riskiest phase — timebox and keep the print pipeline independent of the editor.

Phase 3 — Import/Export (3–4 weeks). DOCX import (mammoth + DOMPurify + style-map to your styles); DOCX export (`docx` library, mapping your nodes → sections/paragraphs/runs). HTML import/export. Set fidelity expectations in UI.

Phase 4 — Collaboration & advanced (6–8 weeks). Yjs + Hocuspocus, presence via Upstash, comments, track changes/suggestions, version history, tables, footnotes, TOC, hyperlinks, spell/grammar, AI assist.

Phase 5 — Objects & polish (ongoing). Floating objects, text wrap, selection pane, align/distribute, accessibility audit, performance hardening, mobile/tablet layouts.

MVP Feature List

Rich text marks (bold/italic/underline/strike/sub/sup/color/highlight/font/size), change case, clear formatting, paragraph alignment/spacing/indents, bulleted/numbered/multilevel lists, Normal+Heading1-3+Quote+Title+Code styles, find/replace (case/whole-word/regex), undo/redo, autosave, inline images, page setup (Letter/A4/Legal, portrait/landscape, margins), **visual pagination**, headers/footers + page numbers, **print + server PDF export**, DOCX import (semantic), HTML import/export, ribbon UI, status bar, zoom, Supabase persistence + auth + RLS sharing.

Advanced Feature List

Real-time collaboration (Yjs), presence cursors, comments, track changes/suggestions mode, version history, templates, tables (pagination-safe), shapes/charts/floating text boxes with text wrap, object arrangement (z-order/align/distribute/group/rotate/flip) + selection pane, footnotes/endnotes, TOC, bookmarks, hyperlinks, section breaks/columns/line numbers/hyphenation, custom + theme-aware styles, DOCX export, spell check (nspell/Hunspell), grammar (LanguageTool), AI writing assistant, voice typing, offline mode, advanced text effects.

Risks & Limitations

1. **Pagination fidelity is the top risk.** No open-source approach matches Word/Docs exactly. Tiptap Pages can hard-lock the editor on a non-splittable block taller than a page; community plugins have placement/perf issues; Paged.js is experimental and Chromium-biased. Mitigation: two-tier (edit vs print) architecture, height caps, import validation, server PDF.
2. **DOCX round-trip is lossy by nature** (see Export/Import strategy). Set expectations.
3. **Floating object text-wrap** cannot fully replicate Word in HTML (`shape-outside` is convex-only; page-anchored floats vs reflow conflict).
4. **Yjs+ProseMirror architectural caveats** (doc re-derivation, rich-text CRDT interleaving) — acceptable but real.
5. **Tiptap Pro/Cloud cost** if you adopt Pages/Conversion/Collab ($49–$999/mo) — mitigated by the MIT-only path.
6. **Solo-builder scope:** this is genuinely a multi-quarter effort; resist building all 12 areas at once.
7. **Advanced text effects (glow/outline)** are screen/PDF-only, won't survive DOCX.
8. **Browser inconsistencies** in clipboard sanitization and `@page` support.

Best Libraries & APIs (named, with licenses)

- **Editor:** Tiptap (MIT core; Pages/Conversion are Pro/subscription) on ProseMirror (MIT).
- **Collaboration:** Yjs (MIT), y-prosemirror (MIT), y-indexeddb (MIT), Hocuspocus server (MIT) or managed Liveblocks/PartyKit.
- **Pagination (edit):** `@tiptap-pro/extension-pages` (subscription) OR `tiptap-extension-pagination` / `@adalat-ai/page-extension` (MIT).
- **Pagination/print (export):** Paged.js (MIT) + Puppeteer/Playwright (Apache-2.0) in headless Chromium.
- **DOCX export:** `docx` by dolanmiu (MIT, current v9.x) — supports sections, page size/margins (twips), headers/footers, styles, images, tables, page numbers, columns; runs client-side (Tiptap's own DOCX export is built on this same library and runs entirely on the client, no JWT/server needed).
- **DOCX import:** mammoth.js (BSD-2-Clause) — semantic mapping (it "intentionally discards font names, font sizes, text colours, and text alignment… clean and style-agnostic"); pair with **DOMPurify** (Apache-2.0/MPL).
- **Server DOCX conversion (optional, higher fidelity):** Pandoc (GPLv2+) run as a separate process to avoid copyleft entanglement; or LibreOffice headless. Tiptap's hosted Conversion REST API is an alternative (App ID + JWT + subscription; headers/footers via REST API require the Team $149 plan).
- **PDF (alt/client):** pdf-lib (MIT) for manipulation; jsPDF (MIT) only for trivial cases; **prefer Paged.js+Puppeteer for fidelity** (client-side `html2canvas`+jsPDF rasterizes text and can't paginate — avoid).
- **Spell check:** nspell (MIT) / nodehun (MIT) + Hunspell dictionaries.
- **Grammar:** LanguageTool (LGPL server / hosted API).
- **Sanitization:** DOMPurify.
- **Suggestions/track changes:** `@handlewithcare/prosemirror-suggest-changes` (MIT) or `prosemirror-suggestion-mode` (MIT). `npm + 2`
- **Stack:** Next.js, TypeScript, Tailwind, Supabase (Postgres+pgvector, Storage, Auth, RLS), Upstash Redis, Vercel.

Performance Strategy

- **Memoize NodeViews** and pass `getPos` (not `pos`) to avoid re-rendering every node after the cursor on each keystroke (the dominant React-ProseMirror perf trap; ProseMirror models positions as integers, so a single insert changes every downstream node's position).

- Rely on ProseMirror's diffing DOM update (it leaves unchanged nodes alone; fixed quadratic update complexity for thousands of child nodes in 2019). Per Emergence Engineering's "Rich Text Editors in Action" stress test, "the number of layout operations grows linearly with nodecount in both editors, though ProseMirror performs about 2-2.5 times more operations than Lexical"; in the same test Lexical's editor "stops consistently around 8200 node counts (~20 minutes)" on memory issues while ProseMirror "usually reached 11500 node counts during the one hour." Net: both slow as node count climbs, so cap document size per logical unit.
- **Pagination cost:** recompute breaks incrementally (only changed blocks), debounce, run in `requestIdleCallback`; never re-fragment the whole doc on every keystroke.
- **Large docs:** consider splitting very large documents into chapters/sections; lazy-render off-screen pages; virtualize thumbnails.
- **Paste:** ProseMirror paste of huge content degrades non-linearly — chunk/parse large pastes off the main thread or progressively. GitHub
- **Move heavy work server-side:** PDF generation, DOCX conversion, grammar — all in Node functions, not the browser.
- **Redis (Upstash)** for presence/rate-limiting; **pgvector** for AI semantic features.

Security Strategy

DOMPurify on all paste/import; mammoth output treated as untrusted (no `externalFileAccess`, always sanitize). Clipboard reads only via explicit user action. File uploads: MIME+magic-byte validation, size caps, zip-bomb guards, isolated conversion functions, reject pathological DOCX (oversized rows). Supabase **RLS** for per-document ACLs (owner/editor/commenter/viewer); server-authoritative permissions. TLS in transit, at-rest encryption. Yjs for conflict-free merges; optimistic concurrency + recovery snapshots for non-collab saves. CSP headers; sandboxed iframes for embeds.

Testing Strategy

- **Unit:** schema transforms, commands (toggle marks, list nesting, case change, sort), style resolution, DOCX mappers.
- **Integration:** import→edit→export round-trip fidelity assertions (golden-file diffs); paste sanitization (XSS payloads must be stripped).
- **E2E (Playwright):** ribbon interactions, pagination behavior, collaboration (two clients converge), print/PDF output snapshots (Paged.js page-image comparison, as Paged.js itself does with Ghostscript).

- **Performance/stress:** automated long-typing loops measuring ScriptDuration/LayoutCount/heap (the Emergence Engineering Lexical-vs-ProseMirror methodology); large-document (100k+ word) load tests.
- **Pagination regression:** assert no infinite-loop on tall blocks; editor page metrics match PDF page metrics.
- **Accessibility:** axe-core, screen-reader passes, keyboard-only operation.
- **Cross-browser:** Chromium/Firefox/Safari matrix; clipboard fallbacks verified.

Export / Import Strategy (DOCX & PDF)

The fidelity ceiling — state it plainly. Round-tripping DOCX through any HTML/JSON-based editor is inherently lossy because the intermediate model doesn't represent every OOXML feature (margins, fonts, positioning, complex tables, tracked changes). This is not a bug you can fix — it's architectural. Pandoc's own docs say it "attempts to preserve the structural elements of a document, but not formatting details such as margin size… conversions from formats more expressive than pandoc's Markdown can be expected to be lossy." CKEditor similarly focuses on the document *body* and must explicitly preserve headers/footers in separate roots to achieve "lossless round-trip." Native-OOXML editors (Apryse, ONLYOFFICE, SuperDoc) avoid this by editing XML directly; you are trading some fidelity for web performance, collaboration, and control. Vendor "perfect fidelity" claims (e.g., Nutrient/PSPDFKit) refer to their proprietary non-HTML layout engines and should be read as marketing, not neutral fact. Apryse Pandoc

DOCX import pipeline: Upload → serverless function → **mammoth.js** with a **style map** (`p[style-name='Heading 1'] => h1`, etc., mapping Word styles to YOUR styles) → **DOMPurify** → HTML → ProseMirror `DOMParser` → JSON. Accept that mammoth "intentionally discards font names, font sizes, text colours, and text alignment" and ignores table border formatting — it produces clean semantic HTML, which is actually ideal for mapping into your style system. For higher fidelity needs, offer a server **Pandoc** or **LibreOffice headless** path (richer but heavier; Pandoc is GPL — run as a separate process to avoid copyleft entanglement; note its server mode can't produce PDFs and won't fetch external resources). Clickcopydoc npm

DOCX export pipeline: ProseMirror JSON → mapper → **`docx` (dolanmiu, MIT)** building Document→Section→Paragraph→TextRun with your styles, page size/margins (twips: 1in=1440), headers/footers, page numbers, images, tables, columns → Packer to Blob/Buffer. This runs **client-side** (the `docx` lib needs no server). If you instead adopt Tiptap's hosted Conversion service: import-docx and the REST endpoints require an App ID + JWT and a subscription (basic conversion is in the Start $49 plan beta; **headers/footers via the REST API require the Team $149 plan**), while their export-docx extension is the exception that runs client-side without credentials — because it is built on the same MIT `docx` library you can use directly and for free. Tiptap Tiptap

Print: for a quick MVP, browser print with `@media print` + `@page`; for production, the same Paged.js pipeline (print the generated paginated HTML).

UX Improvement Recommendations

- **Progressive disclosure:** ribbon for discoverability, but collapse advanced groups by default; surface the 20% of tools used 80% of the time in a Quick Access Toolbar.
- **Contextual intelligence:** mini-toolbar on selection; object/table contextual ribbon tabs (like Word) that appear only when relevant — but never auto-switch the main tab on click (Microsoft guideline).
- **Live, accurate WYSIWYG:** make on-screen page metrics identical to PDF output so "what you see is what you print" actually holds — a top trust factor for a Word competitor.
- **Performance as UX:** keystroke latency under ~16ms; show pagination recompute asynchronously rather than blocking typing.
- **Graceful degradation:** explicit Paste button when clipboard read is blocked; server PDF when client print is unreliable; clear messaging when DOCX import drops formatting.
- **Accessibility-first:** stay on the DOM (don't follow Google to canvas) so screen readers, keyboard nav, and extensions work — this is a competitive advantage, not just compliance.
- **Mobile honesty:** don't cram the ribbon onto phones; give a focused floating-toolbar editing experience.

Final Recommendation

Build on Tiptap + ProseMirror (MIT), persist ProseMirror JSON in Supabase, collaborate with Yjs, paginate visually with Tiptap Pages (or a community MIT plugin to start), and generate print/PDF server-side with Paged.js in headless Chromium. Export DOCX with the MIT `docx` library and import with mammoth.js + DOMPurify, setting honest fidelity expectations. Use a ribbon-primary desktop UI with contextual toolbars and slash-command accelerators, degrading to a floating toolbar on mobile.

This stack uniquely satisfies your two hard constraints — **open-source foundation** and **true pagination** — while fitting your Next.js/TypeScript/Tailwind/Supabase/Upstash/Vercel stack.

Start with the 100%-MIT configuration to control cost and prove the hardest parts (pagination, print) early; adopt Tiptap's paid Pages/Conversion/Collaboration services only when their polish clearly outweighs the subscription. Above all, treat **pagination and print fidelity as the central engineering risk** and sequence the roadmap so you de-risk it in Phase 2, not at the end.